1 Giới thiệu

Đề tài **Dynamic Risk-Aware Multi-UAV Routing** tập trung vào bài toán lập kế hoạch và điều phối tuyến đường cho nhiều UAV trong môi trường có các yêu cầu về thời gian, sức chứa và mức độ rủi ro. Mục tiêu là xây dựng các tuyến đường hiệu quả, đồng thời ưu tiên phục vụ sớm các khách hàng có mức độ rủi ro cao.

Trong bài toán, mỗi khách hàng được xác định bởi vị trí, nhu cầu phục vụ, khung thời gian (*time window*) và thời gian phục vụ. Không gian hoạt động có thể chứa nhiều vùng rủi ro (*risk zones*), trong đó mức độ ảnh hưởng của các vùng này được sử dụng để xác định độ ưu tiên rủi ro (*risk priority*) của từng khách hàng. Vì vậy, việc tối ưu tuyến đường không chỉ hướng đến giảm quãng đường di chuyển mà còn phải cân bằng với yêu cầu giảm độ trễ đối với các khách hàng có mức độ rủi ro cao.

Trong giai đoạn triển khai hiện tại, nhóm tập trung xây dựng mô hình bài toán với tối đa số lượng UAV xác định, các ràng buộc và hàm mục tiêu, đồng thời triển khai hai thuật toán cơ sở (*baseline*) là **NSGA-III** và **Adaptive Large Neighborhood Search (ALNS)**. Các kết quả được đánh giá thông qua tính khả thi, số lượng UAV sử dụng, tổng quãng đường, độ trễ rủi ro (*risk delay*), hàm mục tiêu (*objective*) và thời gian chạy.

Đây là cơ sở để nhóm tiếp tục phân tích độ phức tạp và hiệu năng của các thuật toán, đồng thời phát triển các cơ chế cải tiến trong các giai đoạn nghiên cứu tiếp theo.

2 Đặc tả bài toán

2.1 Tập hợp và tham số

Gọi $N = \{1, 2, \ldots, n\}$ là tập khách hàng và $K = \{1, 2, \ldots, K_{\max}\}$ là tập UAV khả dụng. Mỗi khách hàng $i \in N$ có các thuộc tính:

- Tọa độ: (x_i, y_i);
- Nhu cầu phục vụ: d_i;
- Thời điểm sẵn sàng: e_i;
- Hạn phục vụ: l_i;
- Thời gian phục vụ: s_i^{service};
- Mức độ ưu tiên rủi ro: p_i.

Depot được ký hiệu là nút 0, có tọa độ (x_0, y_0) và thời hạn quay về l_0. Mỗi UAV có sức chứa tối đa Q. Khoảng cách giữa hai nút i và j được tính bằng khoảng cách Euclid:

$$c_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{1}$$

Trong mô hình hiện tại, vận tốc UAV được chuẩn hóa bằng 1, do đó thời gian di chuyển giữa hai nút cũng bằng khoảng cách c_{ij}.

Số UAV tối đa $K_{\max}$ được đọc từ trường `NUMBER` trong bộ dữ liệu Solomon. Với các *instance* đang sử dụng, giá trị này thường là 25, trong khi số khách hàng được xét có thể được giới hạn ở 25 hoặc 100.

2.2 Mức độ rủi ro của khách hàng

Mỗi vùng rủi ro z được mô tả bởi tâm (x_z, y_z) và bán kính r_z. Khoảng cách từ khách hàng i đến tâm vùng rủi ro z là:

$$d_{iz} = \sqrt{(x_i - x_z)^2 + (y_i - y_z)^2} \tag{2}$$

Mức ảnh hưởng của vùng rủi ro z lên khách hàng i được xác định bởi:

$$p_{iz} = \max\left(0, 1 - \frac{d_{iz}}{r_z}\right) \tag{3}$$

Nếu có nhiều vùng rủi ro trong tập các vùng rủi ro Z, độ ưu tiên rủi ro (*risk priority*) của khách hàng i được lấy bằng mức ảnh hưởng lớn nhất:

$$p_i = \max_{z \in Z} p_{iz} \tag{4}$$

Theo đó, $p_i \in [0, 1]$. Giá trị p_i càng lớn cho thấy khách hàng càng gần vùng rủi ro và cần được ưu tiên phục vụ sớm hơn.

2.3 Biến quyết định

Một nghiệm S gồm một tập các tuyến UAV:

$$S = \{R_1, R_2, \ldots, R_K\}, \quad K \leq K_{\max} \tag{5}$$

Mỗi tuyến R_k có dạng:

$$R_k = (0, i_1, i_2, \ldots, i_m, 0) \tag{6}$$

trong đó UAV xuất phát từ depot (nút 0), phục vụ một hoặc nhiều khách hàng, sau đó quay trở lại depot.

Các biến quyết định chính gồm:

- UAV nào được sử dụng;
- Khách hàng nào được gán cho UAV nào;
- Thứ tự phục vụ khách hàng trên mỗi tuyến;
- Thời điểm bắt đầu phục vụ tại mỗi khách hàng.

2.4 Các ràng buộc

2.4.1 Ràng buộc sức chứa

Tổng nhu cầu của các khách hàng trên mỗi tuyến không được vượt quá sức chứa tối đa của UAV:

$$\sum_{i \in R_k} d_i \leq Q, \quad \forall k \in K \tag{7}$$

2.4.2 Ràng buộc phục vụ đầy đủ và duy nhất

Mỗi khách hàng phải được phục vụ đúng một lần:

$$\sum_{k \in K} x_{ik} = 1, \quad \forall i \in N, \tag{8}$$

trong đó $x_{ik} = 1$ nếu khách hàng i thuộc tuyến của UAV k, và bằng 0 trong trường hợp ngược lại.

2.4.3 Ràng buộc cửa sổ thời gian (*Time Window*)

Gọi a_i là thời điểm UAV đến khách hàng i, và t_i là thời điểm bắt đầu phục vụ tại khách hàng i. Khi đó:

$$a_i = t_j + s_j^{\text{service}} + c_{ji}, \tag{9}$$

với j là khách hàng được phục vụ ngay trước i. Thời điểm bắt đầu phục vụ phải thỏa mãn:

$$e_i \leq t_i \leq l_i. \tag{10}$$

Nếu UAV đến trước thời điểm e_i, UAV được phép chờ đến khi cửa sổ thời gian mở.

2.4.4 Ràng buộc hạn quay về depot

Sau khi hoàn thành tuyến, UAV phải quay về depot trước thời hạn l_0:

$$t_{\text{return},k} \leq l_0, \quad \forall k \in K. \tag{11}$$

2.4.5 Ràng buộc số lượng UAV

Số tuyến không rỗng không được vượt quá số UAV khả dụng:

$$K(S) \leq K_{\max}. \tag{12}$$

trong đó $K(S)$ là số tuyến thực tế được sử dụng trong nghiệm S.

2.5 Phạm vi mô hình hiện tại

Trong giai đoạn hiện tại, mô hình sử dụng dữ liệu Solomon VRPTW được mở rộng với các vùng rủi ro tĩnh. Do đó, *risk priority* của khách hàng không thay đổi trong quá trình một lần chạy thuật toán. Mô hình chưa xét các yếu tố động như sự thay đổi bản đồ rủi ro (*risk map*) theo thời gian, thời tiết, mức pin, vùng cấm bay (*no-fly zone*) hoặc sự xuất hiện của các nhiệm vụ mới.

Vì vậy, phiên bản hiện tại được xem là *baseline* tĩnh cho bài toán **Dynamic Risk-Aware Multi-UAV Routing**. Trong các giai đoạn tiếp theo, mô hình có thể được mở rộng bằng cơ chế *rolling horizon*, cập nhật *risk priority* trực tuyến và tái tối ưu tuyến đường khi trạng thái môi trường thay đổi.

3 Thuật toán ALNS

3.1 Tổng quan ALNS

Adaptive Large Neighborhood Search (ALNS) là thuật toán *metaheuristic* dùng để tìm nghiệm cho bài toán định tuyến nhiều UAV có ràng buộc về sức chứa (*capacity*), cửa sổ thời gian (*time window*), hạn quay về depot (*depot deadline*) và giới hạn số lượng UAV.

Mỗi nghiệm gồm nhiều tuyến UAV. Trong mỗi vòng lặp (*iteration*), ALNS thực hiện các bước sau:

1. Chọn một *destroy operator* để loại bỏ một số khách hàng khỏi các tuyến hiện tại.
2. Chọn một *repair operator* để chèn lại các khách hàng bị loại bỏ vào vị trí phù hợp.
3. Kiểm tra tính khả thi của nghiệm mới.
4. So sánh nghiệm mới với nghiệm hiện tại.
5. Cập nhật nghiệm tốt nhất và trọng số của các *operator*.

Thuật toán luôn duy trì nghiệm hiện tại S, nghiệm tốt nhất S_{best} và nghiệm ban đầu S_0.

3.2 Hàm mục tiêu

ALNS tối ưu theo điểm số từ điển (*lexicographical score*):

$$\text{Score}(S) = \big(K(S),\, F(S)\big) \tag{13}$$

trong đó $K(S)$ là số UAV sử dụng và hàm mục tiêu kết hợp $F(S)$ được tính bằng:

$$F(S) = \alpha \cdot \frac{D(S)}{D_{\text{ref}}} + \beta \cdot R(S) \tag{14}$$

Ở đây, $D(S)$ là tổng quãng đường di chuyển, D_{ref} là quãng đường của nghiệm *greedy* ban đầu và $R(S)$ là *risk delay*:

$$R(S) = \frac{\sum_{i \in N} p_i \cdot q_i}{\sum_{i \in N} p_i} \tag{15}$$

với q_i là độ trễ tương đối delay_i tại khách hàng i.

Mục tiêu luôn ưu tiên giảm số UAV trước. Nếu hai nghiệm sử dụng cùng số UAV, thuật toán mới so sánh đến mục tiêu quãng đường - rủi ro $F(S)$.

3.3 Các bước của ALNS

Nghiệm ban đầu S_0 được xây dựng bằng *heuristic greedy*, trong đó các khách hàng được lần lượt chèn vào vị trí khả thi có mức tăng chi phí nhỏ nhất.

Trong bước phá hủy (*destroy step*), ALNS sử dụng các *operator*:

- **Random Removal:** Loại bỏ ngẫu nhiên một số khách hàng.
- **Worst Removal:** Loại bỏ các khách hàng có chi phí đóng góp vào hàm mục tiêu cao nhất.
- **Related Removal:** Loại bỏ các khách hàng có tính chất tương tự nhau (khoảng cách, vị trí...).
- **Route Removal:** Loại bỏ toàn bộ một tuyến UAV để giảm số lượng xe.
- **Risk Priority Removal:** Ưu tiên loại bỏ các khách hàng có rủi ro cao để tái sắp xếp lại vị trí phục vụ sớm hơn.

Trong bước sửa chữa (*repair step*), các *operator* gồm:

- **Greedy Repair:** Chèn lại khách hàng vào vị trí có chi phí phát sinh thấp nhất.

- **Regret-2 Repair:** Ưu tiên chèn khách hàng có sự chênh lệch chi phí lớn giữa vị trí tốt nhất và vị trí tốt thứ hai.
- **Risk Priority Repair:** Ưu tiên chèn các khách hàng có p_i cao vào các vị trí đầu tuyến.

Nghiệm ứng viên (*candidate*) S' chỉ được chấp nhận nếu đảm bảo tính đầy đủ và khả thi. Với ứng viên có cùng số UAV nhưng có mục tiêu $F(S')$ xấu hơn $F(S)$, ALNS có thể chấp nhận theo xác suất của Mô phỏng tôi kim loại (*Simulated Annealing*):

$$P(\text{accept}) = \exp\left(-\frac{F(S') - F(S)}{T}\right) \tag{16}$$

Trọng số của các *operator* được cập nhật dựa trên phần thưởng (*reward*). *Operator* tạo ra nghiệm tốt sẽ tích lũy điểm thưởng cao hơn và có xác suất được chọn thường xuyên hơn trong các vòng lặp tiếp theo.

3.4 Benchmark

Benchmark nhỏ sử dụng các *instance* `Solomon-RISK` với 25 và 100 khách hàng, số vùng rủi ro (*risk zones*) dao động từ 0 đến 5, kết hợp chạy trên nhiều giá trị hạt giống ngẫu nhiên (*seed*) khác nhau. Các chỉ số đánh giá bao gồm:

- Tính khả thi;
- Số UAV sử dụng;
- Tổng quãng đường;
- *Risk delay*;
- Giá trị hàm mục tiêu (*objective*);
- Thời gian thực thi (*runtime*).

Ví dụ, đối với *instance* `C101` gồm 25 khách hàng, 1 vùng rủi ro và thực hiện 1.000 vòng lặp (*iterations*), thuật toán ALNS đạt các kết quả trung bình như trong Bảng 3.4.

3.5 Phân tích độ phức tạp

Gọi n là số lượng khách hàng, q là số lượng khách hàng bị loại bỏ trong mỗi vòng lặp (*iteration*), K là số UAV và I là tổng số vòng lặp.

Việc đánh giá tính khả thi và chi phí của một nghiệm có độ phức tạp thời gian (*time complexity*) là $\mathcal{O}(n)$.

Chỉ số	**Giá trị**
Số UAV	3.00
Tổng quãng đường	239.33
Risk delay	0.0358
Objective	0.6418
Runtime	5.10 giây

Bảng 1: Kết quả đánh giá trung bình của ALNS trên tập dữ liệu C101 (25 khách hàng)

Các *destroy operator* có độ phức tạp khác nhau tùy thuộc vào chiến lược lựa chọn khách hàng (xem Bảng 2).

Bảng 2: Độ phức tạp thời gian của các Destroy Operator

Operator	**Complexity (Implementation)**
Random Removal	$\mathcal{O}(n)$
Route Removal	$\mathcal{O}(n)$
Risk Priority Removal	$\mathcal{O}(qn \log n)$
Worst Removal	$\mathcal{O}(qn^2)$
Related Removal	$\mathcal{O}(qn^3)$

Các *repair operator* là phần tiêu tốn chi phí tính toán lớn nhất do phải duyệt qua nhiều vị trí chèn ứng viên và đánh giá lại toàn bộ các ràng buộc (xem Bảng 3).

Bảng 3: Độ phức tạp thời gian của các Repair Operator

Operator / Heuristic	**Time Complexity**
Greedy Repair	$\mathcal{O}(q^2n(n+K))$
Regret-2 Repair	$\mathcal{O}(q^2n(n+K))$
Risk Priority Repair	$\mathcal{O}(qn(n+K))$

Trong trường hợp xấu nhất (*worst-case*), độ phức tạp tính toán của một vòng lặp ALNS là:

$$\mathcal{O}\big(qn^3 + q^2n(n+K)\big) \tag{17}$$

Do đó, tổng độ phức tạp thời gian của toàn bộ thuật toán ALNS qua I vòng lặp là:

$$\mathcal{O}\big(n^3 + I(qn^3 + q^2n^2)\big) \tag{18}$$

Trong trường hợp q là một hằng số nhỏ ($q \ll n$), độ phức tạp gần đúng của thuật toán sẽ là $\mathcal{O}(I \cdot n^3)$.

Về độ phức tạp không gian (*space complexity*), bộ nhớ sử dụng chủ yếu dành cho ma trận khoảng cách giữa các cặp nút nên có độ phức tạp là $\mathcal{O}(n^2)$.

3.6 Phân tích runtime

Thời gian thực thi (*runtime*) được đo bao gồm thời gian xây dựng nghiệm ban đầu, thực hiện các vòng lặp (*iterations*) và kiểm tra nghiệm cuối cùng. Thời gian đọc và phân tích dữ liệu đầu vào không được tính vào runtime. Mỗi giá trị trong các bảng dưới đây là giá trị trung bình thu được từ 10 hạt giống ngẫu nhiên (*seed*) khác nhau.

3.6.1 Runtime với 25 khách hàng và 1.000 iterations

Bảng 4 và Bảng 5 thể hiện thời gian chạy của thuật toán với các *instance* có 25 khách hàng tương ứng với 0 và 1 vùng rủi ro.

Bảng 4: Runtime với 25 khách hàng, 0 vùng risk (1.000 iterations)

Instance	**Risk zones**	**Runtime trung bình (s)**	**Độ lệch chuẩn (s)**
C101	0	5.02	0.19
C102	0	5.67	0.26
C103	0	6.13	0.29
R101	0	3.76	0.28
R102	0	4.63	0.15
R103	0	4.53	0.25
RC101	0	4.47	0.29
RC102	0	5.34	0.37
RC103	0	5.47	0.18
Trung bình	–	**4.99**	–

Bảng 5: Runtime với 25 khách hàng, 1 vùng risk (1.000 iterations)

Instance	**Runtime trung bình (s)**	**Độ lệch chuẩn (s)**
C101	5.10	0.17
C102	5.42	0.34
C103	6.27	0.29
R101	3.76	0.21
R102	4.46	0.12
R103	4.67	0.25
RC101	4.72	0.30
RC102	5.52	0.37
RC103	5.83	0.25
Trung bình	**5.08**	–

Với 25 khách hàng, runtime nằm trong khoảng từ 3.76 đến 6.27 giây. Chênh lệch giữa 0 và 1 vùng rủi ro là không đáng kể, do *risk priority* chỉ được sử dụng

trong hàm mục tiêu và các *operator*, trong khi kích thước tuyến (*route*) vẫn giữ nguyên.

3.6.2 Runtime với 100 khách hàng, 5 risk zones và 1.000 iterations

Kết quả tổng hợp thời gian chạy theo các nhóm *instance* trên quy mô 100 khách hàng được thể hiện trong Bảng 6.

Bảng 6: Runtime với 100 khách hàng, 5 risk zones (1.000 iterations)

Nhóm instance	**Số instance**	**Runtime trung bình (s)**	**Nhỏ nhất (s)**	**Lớn nhất (s)**
C1	17	218.53	140.75	353.12
R2	23	243.32	149.38	434.14
RC1	16	214.68	155.29	375.02
Toàn bộ	**56**	**227.61**	**140.75**	**434.14**

So với *benchmark* 25 khách hàng, runtime trung bình tăng từ khoảng 5 giây lên 227.61 giây. Như vậy, khi số lượng khách hàng tăng 4 lần (từ 25 lên 100), thời gian thực thi tăng xấp xỉ 45.6 lần.

3.6.3 Nhận xét

Runtime tăng gần như tuyến tính theo số vòng lặp I, do mỗi vòng lặp đều thực hiện một chu trình *destroy-repair* tương tự nhau. Tuy nhiên, runtime tăng phi tuyến tính theo số lượng khách hàng n. Nguyên nhân chính bao gồm:

- Bước *repair* phải duyệt thử nhiều vị trí chèn khả thi;
- Mỗi vị trí chèn đều yêu cầu tính toán và đánh giá lại toàn bộ *route* cũng như nghiệm bộ phận (*partial solution*);
- **Worst Removal** phải tính toán đánh giá chi phí đóng góp của nhiều phương án loại bỏ;
- **Related Removal** trong cài đặt hiện tại phải tính toán lại việc chuẩn hóa độ tương quan (*relatedness normalization*) nhiều lần.

Đối với *benchmark* 25 khách hàng, runtime trung bình đạt khoảng 5 giây cho 1.000 vòng lặp, tương đương khoảng 0.005 giây cho mỗi vòng lặp. Với *benchmark* 100 khách hàng, runtime trung bình tăng lên 227.61 giây, tương đương khoảng 0.228 giây cho mỗi vòng lặp.

Kết quả cho thấy ALNS rất phù hợp với các *instance* có quy mô vừa và nhỏ, nhưng chi phí tính toán tăng đáng kể khi bài toán mở rộng quy mô lớn. Một số hướng nâng cấp để tối ưu runtime bao gồm:

- Bộ nhớ đệm (*cache*) cho độ tương quan *relatedness*;
- Đánh giá tăng đoạn (*incremental evaluation*);
- Chỉ đánh giá lại các phần *route* chịu sự thay đổi trực tiếp.

4 Thuật toán NSGA-III

4.1 Tổng quan

NSGA-III là thuật toán tiến hóa đa mục tiêu (*multi-objective evolutionary algorithm*), được sử dụng để tìm tập nghiệm cân bằng cho bài toán **Dynamic Risk-Aware Multi-UAV Routing**.

Khác với ALNS sử dụng một điểm số từ điển (*lexicographical score*), NSGA-III tối ưu đồng thời ba mục tiêu:

- Giảm số UAV sử dụng;
- Giảm tổng quãng đường di chuyển;
- Giảm độ trễ rủi ro (*risk delay*).

Mỗi cá thể biểu diễn một nghiệm *routing* dưới dạng một hoán vị (*permutation*) của các ID khách hàng. Một bộ giải mã (*decoder*) sẽ chuyển đổi chuỗi hoán vị thành các tuyến UAV khả thi, đồng thời kiểm tra chặt chẽ các ràng buộc về sức chứa (*capacity*), cửa sổ thời gian (*time window*), hạn quay về depot (*depot deadline*) và giới hạn số lượng UAV.

NSGA-III duy trì một quần thể các nghiệm qua nhiều thế hệ. Ở mỗi thế hệ, thuật toán tạo ra các cá thể con bằng các phép lai ghép (*crossover*) và đột biến (*mutation*), sau đó gộp quần thể cha và con để chọn ra quần thể cho thế hệ tiếp theo dựa trên:

1. Sắp xếp không bị trội (*nondominated sorting*);
2. Chuẩn hóa không gian mục tiêu (*objective normalization*);
3. Các hướng tham chiếu (*reference directions*);
4. Phép chọn lọc dựa trên ngách (*niching selection*).

Kết quả cuối cùng thu được là một tập biên Pareto (*Pareto front*), bao gồm tập hợp các nghiệm không bị trội bởi bất kỳ nghiệm nào khác.

4.2 Hàm mục tiêu

Với một nghiệm S, cả ba mục tiêu của NSGA-III đều được tối thiểu hóa.

4.2.1 Số UAV sử dụng

$$f_1(S) = K(S) \tag{19}$$

trong đó $K(S)$ là số tuyến (*route*) không rỗng thực tế được sử dụng trong nghiệm S.

4.2.2 Tổng quãng đường

$$f_2(S) = D(S) = \sum_{k=1}^{K(S)} \sum_{(i,j)\in R_k} c_{ij} \tag{20}$$

Khoảng cách giữa hai nút i và j được tính bằng khoảng cách Euclid:

$$c_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \tag{21}$$

4.2.3 Độ trễ rủi ro (Risk delay)

Đối với khách hàng i, độ trễ tương đối q_i được tính bằng:

$$q_i = \frac{s_i - e_i}{\max(1, l_i - e_i)} \tag{22}$$

trong đó s_i là thời điểm bắt đầu phục vụ khách hàng i, còn $[e_i, l_i]$ là cửa sổ thời gian (*time window*).

Risk delay của toàn bộ nghiệm S được tính bởi:

$$f_3(S) = R(S) = \frac{\sum_{i\in N} p_i q_i}{\sum_{i\in N} p_i} \tag{23}$$

Nếu tổng độ ưu tiên rủi ro $\sum_{i\in N} p_i = 0$, quy ước đặt $R(S) = 0$.

Mức độ ưu tiên rủi ro p_i của khách hàng i được xác định từ tập các vùng rủi ro Z:

$$p_i = \max_{z\in Z} \max\left(0, 1 - \frac{d_{iz}}{r_z}\right) \tag{24}$$

Do đó, bài toán tối ưu đa mục tiêu được phát biểu là:

$$\min \mathbf{F}(S) = \big(K(S),\, D(S),\, R(S)\big) \tag{25}$$

Một nghiệm S_1 được gọi là **trội** (*dominate*) nghiệm S_2 ($S_1 \succ S_2$) nếu:

$$f_j(S_1) \leq f_j(S_2), \quad \forall j \in \{1, 2, 3\} \tag{26}$$

và tồn tại ít nhất một mục tiêu thỏa mãn:

$$f_j(S_1) < f_j(S_2) \tag{27}$$

NSGA-III không gộp ba mục tiêu thành một giá trị vô hướng (*scalar*) duy nhất trong quá trình chọn lọc. Giá trị kết hợp:

$$F(S) = \alpha \frac{D(S)}{D_{\text{ref}}} + \beta R(S) \tag{28}$$

chỉ được tính toán và báo cáo thêm nhằm thuận tiện cho việc so sánh trực tiếp với thuật toán cơ sở ALNS, chứ không phải là mục tiêu duy nhất của NSGA-III.

4.3 Các bước của NSGA-III

4.3.1 Khởi tạo quần thể

Mỗi cá thể ban đầu là một hoán vị (*permutation*) của toàn bộ các ID khách hàng. Một nghiệm *greedy* khả thi được đưa trực tiếp vào quần thể ban đầu. Các cá thể còn lại được tạo bằng cách xáo trộn ngẫu nhiên thứ tự khách hàng.

Sau đó, bộ giải mã (*decoder*) lần lượt chèn từng khách hàng vào các vị trí khả thi của các *route* hiện tại. Ba vị trí có mức tăng quãng đường thấp nhất được giữ lại làm tập lựa chọn ứng viên. Một vị trí trong tập này sẽ được chọn ngẫu nhiên nhằm duy trì tính đa dạng cho quần thể. Nếu không thể chèn khách hàng vào các *route* hiện tại do vi phạm ràng buộc, *decoder* sẽ khởi tạo một *route* mới nếu tổng số *route* chưa vượt quá $K_{\max}$.

4.3.2 Tournament selection

Hai cá thể được chọn ngẫu nhiên từ quần thể để tham gia thi đấu. Cá thể sẽ được ưu tiên chọn nếu nó **trội** (*dominate*) cá thể còn lại. Trong trường hợp hai cá thể không trội lẫn nhau, một cá thể sẽ được chọn ngẫu nhiên.

4.3.3 Order crossover

NSGA-III sử dụng phép lai ghép thứ tự (*Order Crossover - OX*) cho hai hoán vị cha mẹ:

1. Chọn ngẫu nhiên một đoạn chuỗi trên hoán vị của cha/mẹ thứ nhất;

2. Sao chép nguyên vẹn đoạn chuỗi này sang cá thể con;
3. Điền các ID khách hàng còn thiếu vào các vị trí trống theo đúng thứ tự xuất hiện của chúng trong hoán vị của cha/mẹ thứ hai.

Phép toán này bảo đảm mỗi khách hàng xuất hiện đúng một lần trong chuỗi hoán vị của cá thể con.

4.3.4 Swap mutation

Hai vị trí ngẫu nhiên trên chuỗi hoán vị được chọn và tiến hành hoán đổi giá trị cho nhau. Phép đột biến này giúp khám phá các thứ tự phục vụ mới và tránh cho quần thể bị hội tụ sớm vào các cực trị địa phương.

4.3.5 Nondominated sorting

Quần thể cha và quần thể con được gộp lại thành tập hợp ứng viên. Các cá thể sau đó được phân chia thành các tập mặt phẳng không bị trội (*nondominated fronts*):

- **Front 1**: Gồm các nghiệm không bị trội bởi bất kỳ nghiệm nào khác;
- **Front 2**: Gồm các nghiệm chỉ bị trội bởi các nghiệm thuộc Front 1;
- Các **Front** tiếp theo được xây dựng tương tự theo thứ tự phân cấp.

Các *front* lần lượt được đưa vào quần thể của thế hệ mới cho đến khi không thể thêm toàn bộ các cá thể của *front* tiếp theo do vượt quá kích thước quần thể P.

4.3.6 Reference directions và niching

NSGA-III tạo các hướng tham chiếu (*reference directions*) phân bố đều trên không gian ba mục tiêu. Trong cấu hình hiện tại:

- Số lượng mục tiêu: 3;
- Số chia tham chiếu (*reference divisions*): 6;
- Số lượng hướng tham chiếu (*reference directions*): 28;
- Kích thước quần thể (*population size*): $P = 28$.

Các giá trị mục tiêu được chuẩn hóa dựa trên giá trị nhỏ nhất trong quần thể. Mỗi cá thể được gán vào hướng tham chiếu gần nhất dựa trên khoảng cách vuông góc từ điểm mục tiêu đến đường tham chiếu.

Khi *front* cuối cùng không thể đưa toàn bộ vào quần thể mới, cơ chế duy trì ngách (*niching selection*) sẽ ưu tiên chọn:

1. Hướng tham chiếu đang có ít cá thể được gán nhất;
2. Nếu hướng tham chiếu đó chưa có cá thể nào, chọn cá thể có khoảng cách vuông góc gần hướng tham chiếu đó nhất;
3. Nếu hướng tham chiếu đã có cá thể, chọn ngẫu nhiên một cá thể chưa được chọn thuộc hướng đó.

Cơ chế này giúp biên Pareto (*Pareto front*) phân bố đồng đều và rộng khắp hơn trên các hướng đánh đổi giữa số lượng UAV, tổng quãng đường và độ trễ rủi ro.

4.3.7 Thuật toán giả (Pseudo-code)

Cấu trúc từng bước của thuật toán NSGA-III được thể hiện trong Thuật toán 1.

Algorithm 1: Thuật toán NSGA-III cho bài toán Multi-UAV Routing

Input: Thông tin *instance*, cấu hình *config* (gồm P, G, p_c, p_m)
Output: *Pareto front*, nghiệm đại diện S^*, *runtime*

Khởi tạo bộ sinh số ngẫu nhiên từ *seed*;
Xây dựng nghiệm *greedy* khả thi S_0;
Tính D_{ref} từ S_0;
Tạo tập P hướng tham chiếu (*reference directions*);
Khởi tạo quần thể $\mathcal{P}$ gồm P cá thể;
for *generation* $= 1$ **to** G **do**
    $\mathcal{Q} \leftarrow \emptyset$; `/* Tập cá thể con (offspring) */`
    while $|\mathcal{Q}| < P$ **do**
        $\text{parent}_1 \leftarrow$ TournamentSelection($\mathcal{P}$);
        $\text{parent}_2 \leftarrow$ TournamentSelection($\mathcal{P}$);
        child $\leftarrow$ OrderCrossover($\text{parent}_1, \text{parent}_2$)$vixcsut\text{p}_c$;
        child $\leftarrow$ SwapMutation(child) với xác suất p_m;
        routes $\leftarrow$ DecodePermutation(child);
        Đánh giá 3 mục tiêu $\mathbf{F}(\text{child}) = \big(K(\text{child}), D(\text{child}), R(\text{child})\big)$;
        $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{\text{child}\}$
    $\mathcal{R} \leftarrow \mathcal{P} \cup \mathcal{Q}$; `/* Gộp quần thể cha và con */`
    Phân chia $\mathcal{R}$ thành các *nondominated fronts* $(\mathcal{F}_1, \mathcal{F}_2, \dots)$;
    $\mathcal{P}_{\text{new}} \leftarrow \emptyset$;
    Thêm lần lượt từng $\mathcal{F}_i$ vào $\mathcal{P}_{\text{new}}$ cho đến khi không thể thêm trọn vẹn $\mathcal{F}_l$;
    Chuẩn hóa các giá trị mục tiêu của các cá thể trong $\mathcal{R}$;
    Gán các cá thể vào các hướng tham chiếu gần nhất;
    Sử dụng *Niching Selection* để chọn đủ P cá thể từ $\mathcal{F}_l$ vào $\mathcal{P}_{\text{new}}$;
    $\mathcal{P} \leftarrow \mathcal{P}_{\text{new}}$;
$\mathcal{PF} \leftarrow$ Tập các cá thể không bị trội trong quần thể $\mathcal{P}$ cuối cùng;
Chọn nghiệm đại diện S^* từ $\mathcal{PF}$ theo thứ tự ưu tiên từ điển:

$$g(S) = \big(K(S),\, D(S),\, R(S)\big) \tag{29}$$

return $\mathcal{PF}$, S^*, *runtime*;

Nghiệm đại diện S^* trong cài đặt hiện tại được chọn theo thứ tự ưu tiên từ điển $g(S) = \big(K(S), D(S), R(S)\big)$. Tuy nhiên, toàn bộ biên Pareto ($\mathcal{PF}$) vẫn được lưu trữ đầy đủ để phục vụ việc phân tích sự đánh đổi (*trade-off*) giữa các mục tiêu.

4.4 Benchmark

4.4.1 Cấu hình benchmark

Thực nghiệm *benchmark* nhỏ sử dụng ba *instance* `Solomon-RISK` tiêu biểu trên quy mô 25 khách hàng với cấu hình được mô tả trong Bảng 7.

Bảng 7: Cấu hình các instance trong thực nghiệm benchmark

Instance	**Số khách hàng**	**Số vùng risk**	**Số seed**
C101	25	1	5
R101	25	1	5
RC101	25	1	5

Các tham số chạy thuật toán NSGA-III được thiết lập chi tiết trong Bảng 8.

Bảng 8: Cấu hình tham số của thuật toán NSGA-III

Tham số	**Giá trị**
Kích thước quần thể (*Population size*)	28
Số chia tham chiếu (*Reference divisions*)	6
Số thế hệ (*Generations*)	100
Xác suất lai ghép (*Crossover probability*, p_c)	0.9
Xác suất đột biến (*Mutation probability*, p_m)	0.15
Kích thước đấu giá (*Tournament size*)	2
Trọng số quãng đường (α)	0.5
Trọng số rủi ro (β)	0.5

Các chỉ số chất lượng được ghi nhận và báo cáo bao gồm:

- Số lượng UAV sử dụng;
- Tổng quãng đường di chuyển;
- Độ trễ rủi ro (*risk delay*);
- Mục tiêu kết hợp quãng đường - rủi ro (*objective distance-risk*);
- Kích thước tập biên Pareto (*Pareto size*);
- Tính khả thi của nghiệm (*feasibility*);
- Thời gian thực thi (*runtime*).

4.4.2 Kết quả smoke benchmark

Kết quả thực nghiệm sơ bộ đã xác nhận thuật toán hoạt động ổn định với cấu hình kiểm thử nhanh (*smoke test*) gồm 2 thế hệ (*generations*) và hạt giống ngẫu nhiên `seed = 0` (xem Bảng 9).

Bảng 9: Kết quả thực nghiệm smoke benchmark của NSGA-III (2 generations, seed = 0)

Instance	UAV	Distance	Risk delay	Objective	Pareto size	Runtime (s)
C101	3	191.81	0.5232	0.7616	5	0.315
R101	8	632.80	0.4220	0.7110	5	0.278
RC101	6	571.13	0.2424	0.6212	5	0.293

Tất cả các nghiệm thu được trong thực nghiệm *smoke benchmark* đều đảm bảo tính khả thi 100% và phục vụ đầy đủ 25 khách hàng.

Các số liệu trên đại diện cho kết quả chạy kiểm thử trên một *seed*. Bài *benchmark* đầy đủ 5 *seeds* đã được cấu hình trong đường dẫn `configs/benchmark_small.json` và có thể được thực thi thông qua lệnh:

```
cd phase1_NSGA3
uv run python -m nsga3_routing.experiment \
  --config configs/benchmark_small.json \
  --output results/benchmark-small-raw.csv
```

Dữ liệu thô thu được được ghi nhận chi tiết theo từng cặp *instance-seed*. Tệp kết quả tổng hợp (*aggregate*) sẽ tự động tính toán các giá trị trung bình tương ứng cho số lượng UAV, quãng đường, độ trễ rủi ro, hàm mục tiêu kết hợp và thời gian chạy.

4.5 Phân tích độ phức tạp

Để đánh giá độ phức tạp thuật toán NSGA-III, ta quy ước các ký hiệu sau:

- n: Số lượng khách hàng;
- P: Kích thước quần thể (*population size*);
- G: Số lượng thế hệ (*generations*);
- K: Số lượng tuyến UAV (*routes*);
- M: Số lượng mục tiêu (với bài toán này $M = 3$).

4.5.1 Khởi tạo quần thể

Mỗi cá thể được giải mã (*decode*) từ một chuỗi hoán vị. Bộ giải mã thử nhiều vị trí chèn và đánh giá lại *route* sau mỗi phép thử. Trong trường hợp xấu nhất, chi phí giải mã một cá thể là $\mathcal{O}(n^3)$. Do quần thể ban đầu gồm P cá thể, độ phức tạp bước khởi tạo là:

$$T_{\text{initialization}} = \mathcal{O}(Pn^3) \tag{30}$$

4.5.2 Các toán tử biến dị (Variation operators)

Các phép toán chọn lọc đấu giá (*tournament selection*), lai ghép (*crossover*) và đột biến (*mutation*) làm việc trực tiếp trên chuỗi hoán vị nên có chi phí thấp hơn đáng kể so với bộ giải mã:

$$T_{\text{variation}} = \mathcal{O}(Pn) \tag{31}$$

4.5.3 Sắp xếp không bị trội (Nondominated sorting)

Mỗi thế hệ tiến hành xử lý gộp cả quần thể cha và con với kích thước tổng cộng là $2P$. Thuật toán sắp xếp không bị trội nhanh (*Fast nondominated sorting*) cần so sánh từng cặp cá thể với độ phức tạp $\mathcal{O}((2P)^2 M)$. Vì $M = 3$ là một hằng số nhỏ, chi phí này được rút gọn thành:

$$T_{\text{sorting}} = \mathcal{O}(P^2) \tag{32}$$

4.5.4 Chuẩn hóa và duy trì ngách (Normalization và Niching)

Bước chuẩn hóa không gian mục tiêu có chi phí là $\mathcal{O}(PM)$. Việc tính toán khoảng cách vuông góc từ các điểm mục tiêu đến tập các hướng tham chiếu (*reference directions*) có chi phí:

$$T_{\text{niching}} = \mathcal{O}(PHM) \tag{33}$$

trong đó H là số lượng hướng tham chiếu. Với cấu hình mặc định hiện tại ($H = P = 28$), thành phần này có độ phức tạp tương đương:

$$T_{\text{niching}} = \mathcal{O}(P^2) \tag{34}$$

4.5.5 Độ phức tạp trong một thế hệ

Tổng chi phí tính toán chính cho mỗi thế hệ g được xác định bởi:

$$T_{\text{generation}} = \mathcal{O}(Pn^3 + P^2) \tag{35}$$

4.5.6 Độ phức tạp tổng thể của NSGA-III

Thực thi qua G thế hệ, tổng độ phức tạp thời gian (*time complexity*) của toàn bộ thuật toán NSGA-III là:

$$T_{\text{NSGA-III}} = \mathcal{O}\Big(Pn^3 + G(Pn^3 + P^2)\Big) \tag{36}$$

Nếu kích thước quần thể P được cố định ở một giá trị nhỏ, độ phức tạp thời gian gần đúng theo số lượng khách hàng n và số thế hệ G sẽ là:

$$T_{\text{NSGA-III}} = \mathcal{O}(G \cdot n^3) \tag{37}$$

4.5.7 Độ phức tạp không gian (Space complexity)

Bộ nhớ sử dụng trong suốt quá trình chạy thuật toán bao gồm:

- Ma trận khoảng cách (*distance matrix*): $\mathcal{O}(n^2)$;
- Quần thể cha và quần thể con (*population & offspring*): $\mathcal{O}(Pn)$;
- Tập các hướng tham chiếu (*reference directions*): $\mathcal{O}(PM)$;
- Các tuyến ứng viên tạm thời (*candidate routes*): Tối đa khoảng $\mathcal{O}(n^2)$.

Do đó, độ phức tạp bộ nhớ tổng thể được biểu diễn bởi:

$$M_{\text{NSGA-III}} = \mathcal{O}(n^2 + Pn) \tag{38}$$

4.6 Phân tích runtime

Thời gian thực thi (*runtime*) được đo bằng hàm `time.perf_counter()` trong hàm `run_nsga3`. Khoảng thời gian đo bao gồm toàn bộ các bước:

- Xây dựng nghiệm ban đầu;
- Khởi tạo quần thể;
- Tạo thế hệ con (*offspring*);
- Sắp xếp không bị trội (*nondominated sorting*);
- Ánh xạ hướng tham chiếu (*reference-direction association*);
- Chọn lọc dựa trên ngách (*niching selection*);
- Đánh giá tập biên Pareto (*Pareto front*) cuối cùng.

Thời gian phân tích (*parse*) tệp dữ liệu đầu vào không được tính vào runtime của NSGA-III.

Thời gian chạy thực tế phụ thuộc chủ yếu vào các yếu tố:

- Số lượng khách hàng (n);
- Kích thước quần thể (P);
- Số lượng thế hệ (G);
- Số lượng tuyến (*routes*) trung gian;
- Số phép thử chèn (*insertion*) trong bộ giải mã (*decoder*);
- Cấu hình phần cứng và hạt giống ngẫu nhiên (*seed*).

Kết quả thực nghiệm kiểm thử nhanh (*smoke benchmark*) cho thấy runtime của 2 thế hệ nằm trong khoảng từ 0.278 đến 0.315 giây. Runtime trung bình của ba *instance* thử nghiệm là:

$$\bar{t}_{\text{smoke}} = \frac{0.315 + 0.278 + 0.293}{3} \approx 0.295 \text{ giây} \tag{39}$$

Thời gian chạy trung bình trên mỗi thế hệ (*generation*) xấp xỉ:

$$t_{\text{per_gen}} = \frac{0.295}{2} \approx 0.148 \text{ giây} \tag{40}$$

Kết quả này dùng để kiểm tra tính đúng đắn trong hoạt động của chương trình trên một *seed*. Khi thực hiện bài *benchmark* đầy đủ trên s giá trị *seed* khác nhau, runtime trung bình $\bar{t}$ được tính theo công thức:

$$\bar{t} = \frac{1}{s} \sum_{j=1}^{s} t_j \tag{41}$$

Độ lệch chuẩn của runtime (σ_t) được xác định bởi:

$$\sigma_t = \sqrt{\frac{1}{s-1} \sum_{j=1}^{s} (t_j - \bar{t})^2} \tag{42}$$

Runtime tăng gần như tuyến tính theo số lượng thế hệ G, do mỗi thế hệ đều phải khởi tạo, biến dị và đánh giá lại một quần thể con mới. Tuy nhiên, thời gian chạy sẽ tăng phi tuyến tính theo số lượng khách hàng n do bộ giải mã phải kiểm tra nhiều vị trí chèn ứng viên và đánh giá lại các ràng buộc trên từng tuyến.

So với thuật toán ALNS, NSGA-III chịu thêm một số chi phí tính toán đặc thù bao gồm:

- Duy trì đồng thời cả quần thể cha (*population*) và quần thể con (*offspring*);
- Thuật toán phân loại không bị trội (*nondominated sorting*);
- Chuẩn hóa không gian mục tiêu (*normalization*);
- Ánh xạ khoảng cách tới các hướng tham chiếu (*association*);
- Phép chọn lọc duy trì đa dạng (*niching selection*).

Đổi lại, NSGA-III cung cấp một tập hợp đa dạng các nghiệm Pareto trong một lần chạy duy nhất, cho phép nhà quản lý phân tích trực quan sự đánh đổi (*trade-off*) giữa số lượng UAV, tổng quãng đường di chuyển và độ trễ rủi ro (*risk delay*) thay vì chỉ nhận về một nghiệm vô hướng đơn lẻ như ALNS.